```
!pip install qiskit qiskit-aer
```

1. Create a single-qubit circuit applying the Hadamard gate

```
from qiskit import QuantumCircuit

# Create a quantum circuit with 1 qubit and 1 classical bit

qc = QuantumCircuit(1, 1)

# Apply Hadamard gate to put the qubit in superposition

qc.h(0)

# Draw the circuit

print(qc.draw())
```

2. Measure the qubit and plot the histogram of results

```
from qiskit import QuantumCircuit

from qiskit_aer import AerSimulator

from qiskit.visualization import plot_histogram

# 1. Create circuit

qc = QuantumCircuit(1, 1)

qc.h(0)

qc.measure(0, 0)

# 2. Simulate the circuit

simulator = AerSimulator()

job = simulator.run(qc, shots=1024)
```

```
result = job.result()

counts = result.get_counts()

print("Measurement Results:", counts)

# 3. Plot histogram

display(plot_histogram(counts))
```

3. Build a circuit to generate a Bell state

```
from qiskit import QuantumCircuit

# Create a quantum circuit with 2 qubits and 2 classical bits

qc = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit

qc.h(0)

# Apply CNOT gate (Control: 0, Target: 1)

qc.cx(0, 1)

# Measure both qubits

qc.measure([0, 1], [0, 1])

print(qc.draw())
```

4. Submit a Bell state circuit to a quantum simulator (IBM)

```
from qiskit import QuantumCircuit

from qiskit_aer import AerSimulator

# Create Bell state circuit

qc = QuantumCircuit(2, 2)

qc.h(0)
```

```
qc.cx(0, 1)

qc.measure([0, 1], [0, 1])

# Using Qiskit Aer Simulator (Local IBM simulator backend)

simulator = AerSimulator()

# Run the job on the simulator

job = simulator.run(qc, shots=1000)

result = job.result()

counts = result.get_counts(qc)

print("Simulation Results from IBM Simulator backend:", counts)
```

5. Implement a multi-qubit circuit with CNOT and SWAP gates

```
from qiskit import QuantumCircuit

# Create a circuit with 3 qubits and 3 classical bits

qc = QuantumCircuit(3, 3)

# Apply X gate on qubit 0 to set state to |1>

qc.x(0)

# Apply CNOT gate (control: 0, target: 1)

qc.cx(0, 1)

# Apply SWAP gate between qubit 1 and qubit 2

qc.swap(1, 2)

# Measure all qubits

qc.measure([0, 1, 2], [0, 1, 2])

print(qc.draw())
```

6. Implement and run the Deutsch-Jozsa algorithm on a balanced and constant function

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
def deutsch_jozsa(oracle_type, n=2):
    qc = QuantumCircuit(n + 1, n)
    # Initialize auxiliary qubit to |1>
    qc.x(n)
    # Apply Hadamard gates to all qubits
    for qubit in range(n + 1):
        qc.h(qubit)
    # Apply Oracle
    if oracle_type == "balanced":
        # Balanced Oracle using CNOT gates
        for qubit in range(n):
            qc.cx(qubit, n)
    elif oracle_type == "constant":
        # Constant oracle does nothing to input/output relation
        pass
    # Apply Hadamard gates again to input qubits
    for qubit in range(n):
        qc.h(qubit)
    # Measure input qubits
    for qubit in range(n):
        qc.measure(qubit, qubit)
```

```
    return qc

# Test Balanced Oracle

qc_balanced = deutsch_jozsa("balanced", n=2)

simulator = AerSimulator()

result = simulator.run(qc_balanced, shots=1024).result()

print("Balanced Oracle Results (00 means constant, others mean balanced):",
result.get_counts())
```

7. Implement and run Grover's algorithm to search for a marked state

```
from qiskit import QuantumCircuit

from qiskit_aer import AerSimulator

# 2-qubit Grover's algorithm searching for state |11>

qc = QuantumCircuit(2, 2)

# 1. Superposition

qc.h([0, 1])

# 2. Oracle for state |11>

qc.cz(0, 1)

# 3. Diffusion Operator

qc.h([0, 1])

qc.x([0, 1])

qc.h(1)

qc.cx(0, 1)

qc.h(1)

qc.x([0, 1])

qc.h([0, 1])
```

```
# 4. Measurement
qc.measure([0, 1], [0, 1])
print(qc.draw())
# Run simulation
simulator = AerSimulator()
counts = simulator.run(qc, shots=1024).result().get_counts()
print("Grover Search Results (Target |11>):", counts)
```

8. Implement and run Quantum Fourier Transforms and Shor's algorithm (QFT Part)

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
import numpy as np
def qft(circuit, n):
    """Implements the QFT on n qubits"""
    for j in range(n):
        circuit.h(j)
        for k in range(j + 1, n):
            circuit.cp(np.pi / float(2**(k - j)), k, j)
# 3-qubit QFT Circuit
n = 3
qc = QuantumCircuit(n, n)
# Apply QFT function
qft(qc, n)
# Measure
```

```
qc.measure(range(n), range(n))

print(qc.draw())

# Run simulation

simulator = AerSimulator()

result = simulator.run(qc, shots=1024).result().get_counts()

print("QFT Results:", result)
```

Do it again:

2. Measure & Plot Histogram (Chota Version)

```
from qiskit import QuantumCircuit

from qiskit_aer import AerSimulator

from qiskit.visualization import plot_histogram

qc = QuantumCircuit(1, 1)

qc.h(0)

qc.measure(0, 0)

# Direct simulate and plot

counts = AerSimulator().run(qc).result().get_counts()

display(plot_histogram(counts))
```

6. Deutsch-Jozsa Algorithm (Chota Version)

```
from qiskit import QuantumCircuit

from qiskit_aer import AerSimulator
```

```
# Simple 2-qubit Balanced Deutsch-Jozsa Circuit
qc = QuantumCircuit(2, 1)
qc.x(1)           # Auxiliary qubit to |1>
qc.h([0, 1])      # Hadamard on both
qc.cx(0, 1)       # Oracle (CNOT for balanced)
qc.h(0)           # Hadamard on input
qc.measure(0, 0)    # Measure input qubit
print(qc.draw())
```

8. QFT / Shor's Algorithm Part (Chota Version)

```
from qiskit import QuantumCircuit
import numpy as np
# 2-qubit QFT
qc = QuantumCircuit(2)
qc.h(0)
qc.cp(np.pi/2, 1, 0)  # Controlled phase gate
qc.h(1)
print(qc.draw())
```